



**Hochschule
Bonn-Rhein-Sieg**

*University
of Applied Sciences*

Fachbereich Informatik
Department of Computer Sciences

Praktikumsbericht

Informatik B.Sc.

**Konzeption und Entwicklung einer Software
zur Erstellung sicherheitsbezogener Blockdiagramme
gemäß EN ISO 13849 mit Anbindung an SISTEMA 3.0**

vorgelegt von
Rejk Böning

Betreuer Hochschule : Prof. Dr. Stefan Böhmer
Betreuer DGUV : Andy Lungfiel
Eingereicht am : 10.7.2026 (Vorläufig)

Inhaltsverzeichnis

1. Der Weg zum Unternehmen	1
2. Das Unternehmen	1
2.1. Unternehmensdaten	1
2.2. Unternehmensaufgaben	1
3. Aufgabe	2
3.1. Allgemeine Aufgabenbeschreibung	2
3.2. Projektvorgaben	3
4. Umsetzung	5
4.1. Auswahl der Entwicklungstechnologien	5
4.2. Implementierung	6
4.3. Architektur der Anwendung	10
4.3.1. Slint als View im MVC Kontext	10
5. Bezug zum bisherigen Studium	14
6. Fachliche und persönliche Erfahrungen	14
7. Bibliografie	15

Abbildungsverzeichnis

Tabelle 1	„Übersicht über die Unternehmensdaten“	1
Abbildung 1	„Auszug aus dem Jahresbericht 2024 [1] S.4“	2
Abbildung 2	Auszüge eines Schaltplans und seines Blockdiagramms aus dem IFA- Report 2017 (TODO: QUELLE HIER EINFÜGEN) S.120-121	3
Tabelle 2	Durch SISTEMA beeinflusste Rahmenbedingungen	4
Abbildung 3	„Dummy“ Graph-Editor in Rust und Slint als Test der Entwicklungswerkzeuge implementiert	5
Tabelle 3	Projekt Crate Übersicht	6
Abbildung 4	Graphbasierte der Übersicht der „bedit“ crate	7
Abbildung 5	Graphbasierte der Übersicht der „sistema“ crate	8
Abbildung 6	Graphbasierte der Übersicht der „delta“ crate	9
Abbildung 7	Graphbasierte der Übersicht der „sbe_sis“ crate	10
Abbildung 8	Graphbasierte der Übersicht der „bedit_app“ crate	10
Beispiel 1	Beispiel Slint-Komponente mit einem Callback und einem Property	11
Beispiel 2	Pseudo Rust-Code für einen Controller zu Beispiel 1	12
Abbildung 9	Safety Block Designer (SBE) Architektur Übersicht	13

Abkürzungsverzeichniss

ABI	Application Binary Interface
DGUV	Deutsche Gesetzliche Unfallversicherung
DSL	Domain Specific Languge
IFA	Institut für Arbeitsschutz
MVC	Model-View-Controller
PL	Performance Level
QOL	Quality of Live
SBE	Safety Block Designer

1. Der Weg zum Unternehmen

Im Februar 2026 habe ich mich durch eine Initiativbewerbung bei der Deutschen Gesetzlichen Unfallversicherung (DGUV) im Institut für Arbeitsschutz (IFA) auf eine Praxisprojektstelle beworben. Erfreulicherweise konnte mir die DGUV mehrere Projekte anbieten, die für ein Praxisprojekt geeignet wären. In Absprache mit der DGUV wurde entschieden, dass ich an einer neuen Software zur Erstellung sicherheitsbezogener Blockdiagramme arbeiten könnte, auf Basis deren sich auch eine Abschlussarbeit schreiben lassen würde.

Auf die DGUV bin ich durch Ihre Nähe zur Hochschule und positive Arbeitserfahrungen von Verwandten aufmerksam geworden. Schlussendlich überzeugt, mein Praxisprojekt bei der DGUV absolvieren zu wollen, hat mich sowohl das Angebot, dort meine Bachelorarbeit schreiben zu können, als auch die interessanten und netten Präsentationen zu den Projektmöglichkeiten, die ich dort erhielt.

2. Das Unternehmen

2.1. Unternehmensdaten

Die Deutsche Gesetzliche Unfallversicherung (DGUV) ist der Spitzenverband der gewerblichen Berufsgenossenschaften und der Unfallversicherungsträger der öffentlichen Hand [2]. Dieser wurde am 01.06.2007 durch die Zusammenlegung des Hauptverbandes der gewerblichen Berufsgenossenschaften e. V. (HVBG) und des Bundesverbandes der Unfallkassen e. V. (BUK) gegründet [3]. Der HVBG wurde 1887 gegründet während der BUK 1936 gegründet wurde.

Deutsche Gesetzliche Unfallversicherung (DGUV)

Vollständiger Name	Deutsche Gesetzliche Unfallversicherung e.V. (DGUV)
Standort Berlin	Glinkastraße 40, 10117 Berlin
Standort Bonn	Alte Heerstraße 111, 53757 Sankt Augustin
Hauptsitz	Berlin
Gründung	01.06.2007

Tabelle 1: „Übersicht über die Unternehmensdaten“

2.2. Unternehmensaufgaben

Die Aufgaben des IFA sind extrem vielfältig und ausschweifend¹, trotzdem schafft es der IFA-Jahresbericht für das Jahr 2024 die Aufgaben des IFA in wenigen Sätzen zu beschreiben:

„Das IFA unterstützt die Präventionsarbeit der Berufsgenossenschaften und Unfallkassen. Seine Mitarbeitenden forschen, analysieren und beraten zu naturwissenschaftlich-technischen Fragen des Arbeits- und Gesundheitsschutzes. Darüber hinaus bietet das Institut Produktprüfungen und -zertifizierungen an.“ (IFA Jahresbericht 2024 [1], S.4)

Das IFA hat 5 wissenschaftliche, technische Hauptarbeitskapazitäten; namentlich sind das die Analytik(1), Prüfung und Zertifizierung(2), (technische) Infrastruktur(3), Entwicklung und Untersuchung(4) und zu guter Letzt die Beratung(5) [1]. Dabei übernimmt das IFA die Aufgabe Gefährdungen des Arbeitsschutzes zu erforschen, Schutzmaßnahmen zu entwickeln und Richtlinien diesbezüglich zu erarbeiten und/oder bei deren Prüfung zu assistieren.

¹Ich beschränke mich darauf nur die Aufgaben des IFA zu beschreiben, da dies das Institut der DGUV ist in welchem mein Praxisprojekt stattgefunden hat.

Die Tätigkeiten, an die ich im Rahmen meines Praxisprojekts am meisten herangeführt wurde, waren die Arbeiten im Zusammenhang mit der Sicherheitsfachgrundnorm DIN EN ISO 13849-1. Diese normt die Anforderungs- und Maßnahmenenerhebungen für Maschinensteuerungen bezüglich ihrer funktionalen Sicherheit.

Arbeitskapazitäten nach Aufgabenbereichen 2024

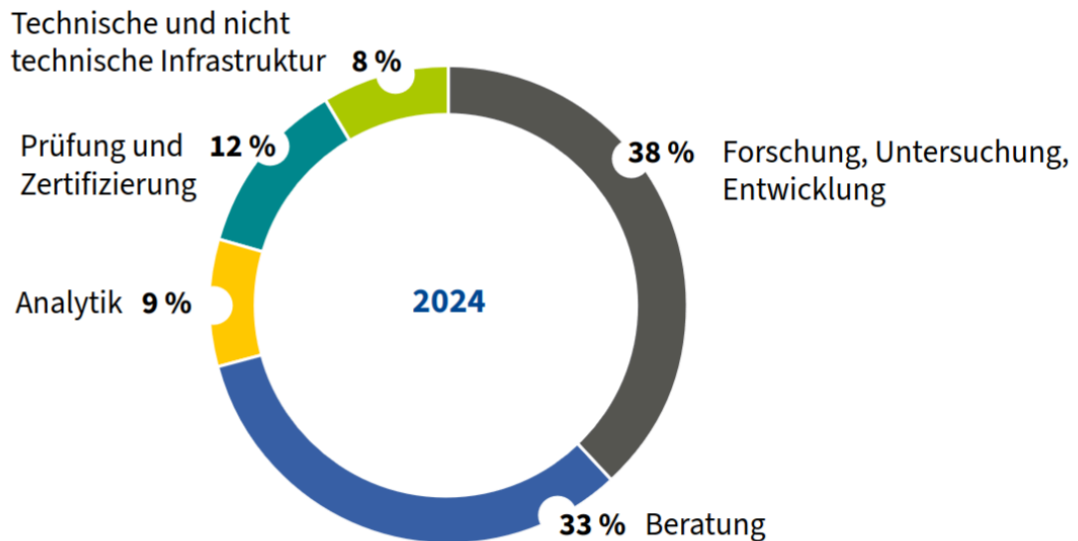


Abbildung 1: „Auszug aus dem Jahresbericht 2024 [1] S.4“

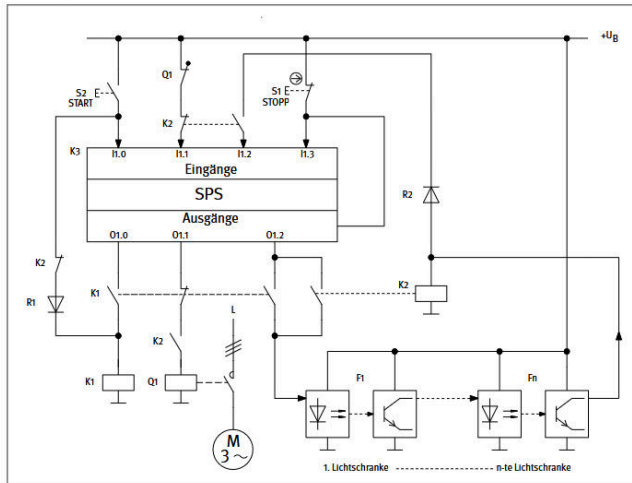
3. Aufgabe

3.1. Allgemeine Aufgabenbeschreibung

Die Norm DIN EN ISO 13849-1 beschreibt Anforderungen an sicherheitsbezogene Teile von Maschinensteuerungen und deren Bewertung hinsichtlich eines sogenannten Performance Level (PL). Für die Bestimmung dieses PL bietet das IFA bereits SISTEMA an, eine umfangreiche Anwendung für die strukturelle Modellierung, Dokumentierung und anschließende PL bestimmung von Maschinensteuerungen.

Teil der Vorarbeit zur berechnung und dokumentation der Sicherheitsfunktionen einer Steuerung in SISTEMA ist die Transformation von einem elektrischen Schaltplan in ein sicherheitsbezogenes Blockdiagramm.

Schaltplan



Sicherheitsbezogenes Blockdiagramm

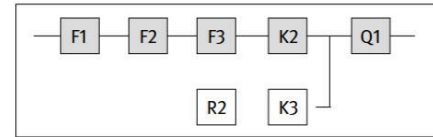


Abbildung 2: Auszüge eines Schaltplans und seines Blockdiagramms aus dem IFA-Report 2017
(**TODO: QUELLE HIER EINFÜGEN**) S.120-121

Dieser Transformationsschritt stellt in der Praxis eine besondere Herausforderung dar. Während Schaltpläne die physikalische Verschaltung von Komponenten beschreiben, bildet das sicherheitsbezogene Blockdiagramm die logische Struktur der Sicherheitsfunktion mit ihren Funktions-, Test- und Diagnosepfaden ab. Die Ableitung erfolgt bislang überwiegend manuell und erfordert umfangreiches Expertenwissen.

Um den Prozess der Blockdiagrammerstellung angenehmer und weniger Fehleranfällig zu gestalten soll als Ziel meines Praxisprojekts ein Anwendungsprototyp erstellt werden. Dieser soll es Nutzern erlauben, grafisch geführt, sicherheitsbezogene Blockdiagramme zu entwerfen und zu dokumentieren. Die Diagramme und Dokumentation sollen von der Software als SISTEMA-Projekt exportiert werden können.

3.2. Projektvorgaben

Der Anwendungsprototyp SBE soll als Komplementärsoftware zu SISTEMA entwickelt werden. Das bedeutet, dass sich der SBE möglichst nahtlos in einem bereits bestehenden Workflow mit SISTEMA integrieren lassen soll. Er muss deshalb für die bestehenden SISTEMA-Nutzer leicht verständlich, nützlich und zugänglich sein. Das Achten auf „SISTEMA-Kompatibilität“ ist für den SBE besonders wichtig, da rund 70 bis 80% der globalen Normanwender SISTEMA verwendet [4].

Diese enge bindung an SISTEMA gibt dem SBE eine Reihe an Rahmenbedingungen vor.

Bedingung	Beschreibung	Begründung
1) Sistemaprojekt Import/Export	SBE soll in der Lage sein Sistemaprojekte zu laden und zu exportieren	Der Im- und Export von Sistemaprojekten ist eine einfache und intuitive Art und Weise um die zwei Anwendungen miteinander zu verbinden. Bestehende Nutzer haben ihre Projekte bereits im SISTEMA-Format gespeichert und würden so eine Funktionen vom SBE erwarten.
2) SBE muss Übersetzbar sein	Die Anwendung soll in alle von SISTEMA unterstützten Sprachen übersetzbar sein. Diese Übersetzungen sollen zur Laufzeit dargestellt werden. Zudem soll die Systemsprache des Nutzers erkannt und standardmäßig in SBE eingestellt sein.	Da SISTEMA international verwendet wird ist eine gute Unterstützung von verschiedenen Sprachen unumgänglich.
3) Selbe Plattformunterstützung wie SISTEMA	Die Anwendung soll als Desktopanwendung für Windows 10 und 11 zur Verfügung stehen. Es sollen keine zusätzlichen Laufzeitumgebungen oder Abhängigkeiten für die Inbetriebnahme erforderlich sein.	Der SBE soll mindestens auf den selben Plattformen wie SISTEMA zur Verfügung stehen, damit die Anwendung allen SISTEMA-Nutzern zur Verfügung stehen kann.

Tabelle 2: Durch SISTEMA beeinflusste Rahmenbedingungen

Während diese Rahmenbedingungen eingehalten werden soll der SBE folgende Funktionen bereitstellen:

- Ein Projekt soll mehrere Sicherheitsfunktionen und eine Projektdokumentation enthalten können. Dieses Projekt soll in SBE erstellt, gespeichert, geladen und bearbeitet werden können. Das selbe Projekt wäre auch das Ziel für einen Import/Export zwischen SISTEMA und SBE.
- Für jede Sicherheitsfunktion soll der Nutzer in der Lage sein Dokumentation und ein Blockdiagramm zu erstellen. Die Blockdiagramme sollen normkonform sein und aus Teilsystemen mit Bauteilen bestehen die der Nutzer ebenfalls pro Sicherheitsfunktion verwalten kann.
- SBE soll zusätzlich zum Import/Export von SISTEMA Projekten eine „nahtlose“ Zusammenarbeit mit SISTEMA ermöglichen. Es ist geplant, dass der SBE nicht nur als Eigenständige Anwendung verfügbar ist, sondern auch als eine Art Plugin in SISTEMA.
- Das Bearbeiten der Blockdiagramme soll intuitiv und zeitsparend sein. Dazu sollte die Bearbeitung der Diagramme zum einem großen Teil mittel „Drag’n Drop“ möglich sein.
- Um das Erstellen der Diagramme Workflowtechnisch von dem Dokumentieren der in den Diagrammen enthaltenen Bauteile zu trennen soll ein Nutzer in der Lage sein, seine Bauteile in einer Bauteilbibliothek anzulegen und zu dokumentieren bevor er beginnt das Diagramm zu erstellen.
- Das Programm soll eine angenehme Menüführung haben und Anwenderfreundlich sein. Das beinhaltet eine „schnelle“, moderne Benutzeroberfläche und Quality of Live (QOL) Features wie etwa eine Undo/Redo Funktionalität.

4. Umsetzung

4.1. Auswahl der Entwicklungstechnologien

Da der SBE eine von grundauf neu entwickelte Anwendung sein wird, musste ich die Entscheidung welche Programmiersprache(n), Frameworks und andere Entwicklungswerkzeuge ich für die Entwicklung verwenden möchte selbst treffen.

In Anbetracht der Rahmenbedingungen, dass der SBE keine zusätzlichen Installationen von Laufzeitumgebungen oder Abhängigkeiten vom Endanwender erfordern soll UND als Plugin oder Bibliothek für SISTEMA zur Verfügung stehen soll, habe ich mich gegen eine Sprache wie etwa Python oder Java entschieden. Beide Sprachen sind zwar populäre Sprachen für die Desktopanwendungsentwicklung, erfordern aber die Installation oder das Bündeln von Laufzeitumgebungen oder dem Pythoninterpreter. Zusätzlich wäre es aufwändiger den SBE für SISTEMA als Plugin oder Bibliothek verfügbar zu machen.

Da ich die Einbettung des SBE in SISTEMA am liebsten durch eine Shared-Library gelöst hätte, habe ich mich entschieden eine Kompilierte Sprache wie C++ oder Rust zu verwenden. Das würde es mir auch erlauben alle Ressourcen und Abhängigkeiten die der SBE verwenden wird, mittels dem Linker in der ausführbaren Programmdatei (Binary) ein zu betten um so der Rahmenbedingung 3) noch mehr entgegen zu kommen.

Schlussendlich ist meine Auswahl auf Rust gefallen, da ich durch vorherige Arbeit und Hobby-Projekte sehr gute Erfahrungen mit Rust und seinem Build- und Ecosystem gemacht habe und mir sicher war, alle Rahmenbedingungen und Features zügig und zuverlässig in Rust implementieren zu können. Für einen Rust-Kompatiblen UI-Framework ist meine Wahl dabei auf Slint [5] gefallen, hauptsächlich wegen der guten unterstützung von Übersetzungen in der UI und einer Reihe an Tests die ich zu Anfang der Entwicklung mit Slint implementiert habe².

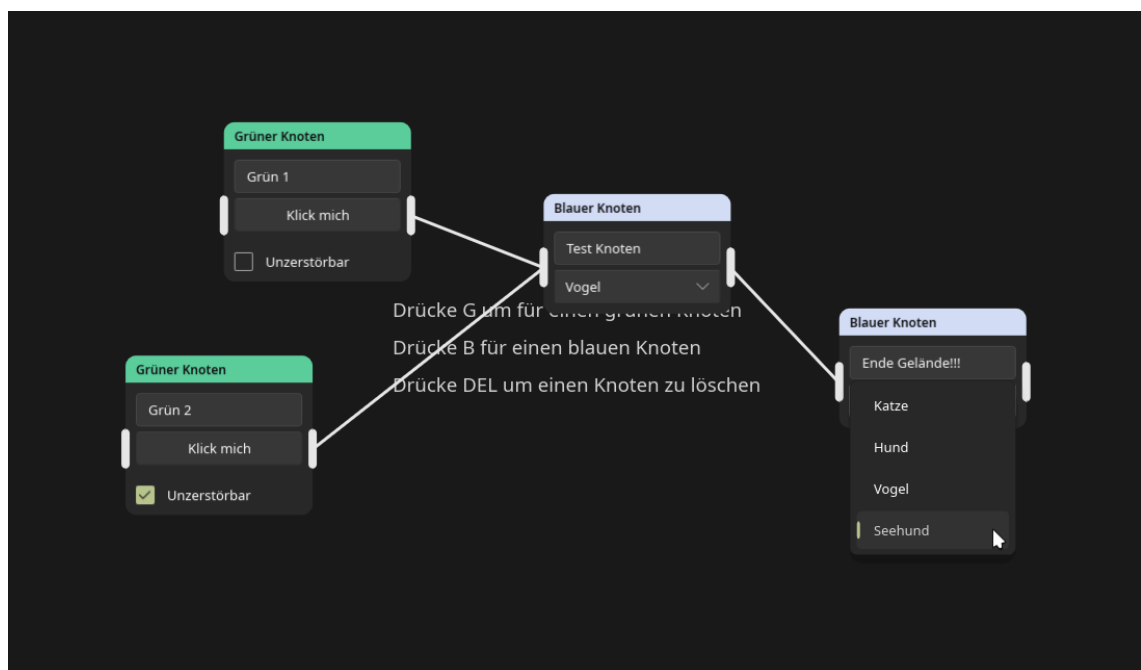


Abbildung 3: „Dummy“ Graph-Editor in Rust und Slint als Test der Entwicklungswerkzeuge implementiert

²Um den Bericht kurz zu halten werde ich auf diese Tests nicht näher eingehen.

4.2. Implementierung

Da eine komplette beschreibung der Implementierung über den Rahmen dieses Berichts hinausgehen würde, werde ich einen kurzen Übertblick über die Projektstrucktur und Architektur geben und dann die Implementation einiger ausgewählter Features näher erläutern.

Das Projekt ist in 5 Crates³, mit folgenden Aufgaben aufgeteilt

Crate	Typ	Aufgabe
bedit	Library	Rust library crate für die SBE Awendung. Beinhaltet die Benutzeroberfläche und das Backend von SBE.
bedit_app	Binary	Binary crate für die SBE Awendung. Konsumiert die bedit crate und macht sie Als „Standalone“ Anwendung verfügbar.
sbe_sis	Dynamic Library	Ein C-Kompatibler Wrapper für die bedit Crate. Macht die bedit library für andere Anwendungen als Shared-Library verfügbar. Da bedit selbst kein C-Kompatibles (oder überhaupt stabiles) Application Binary Interface (ABI) hat, braucht es eine schmale Schnittstelle, die von anderen Anwendungen durch normales Linken oder ein FFI-Interface konsumiert werden kann.
sistema	Library	Stellt einen SISTEMA-Projektdatei-Parser bereit, der (beidseitig) zwischen dem SISTEMA-Speicherformat (.ssm) und einer Rust Datenstrucktur konvertieren kann.
delta	Library	Implementation eines Generischen Undo/Redo Systems, dass die Differenz zwischen 2 Objekten einer Datenstrucktur erkennen und versionieren kann. Das System ist vollständig serialisierbar um z.B. das automatische speichern von wiederherstellen von Sicherungskopien zu ermöglichen.

Tabelle 3: Projekt Crate Übersicht

³Eine Rust-„Crate“ wäre in anderen Sprachen eine Programmbibliothek

Eine grobe veranschaulichung der Struckturen jeder Crate bieten folgende Graphen. Höher auflösende Versionen dieser Graphen liegen dem Digitalen Bericht bei⁴

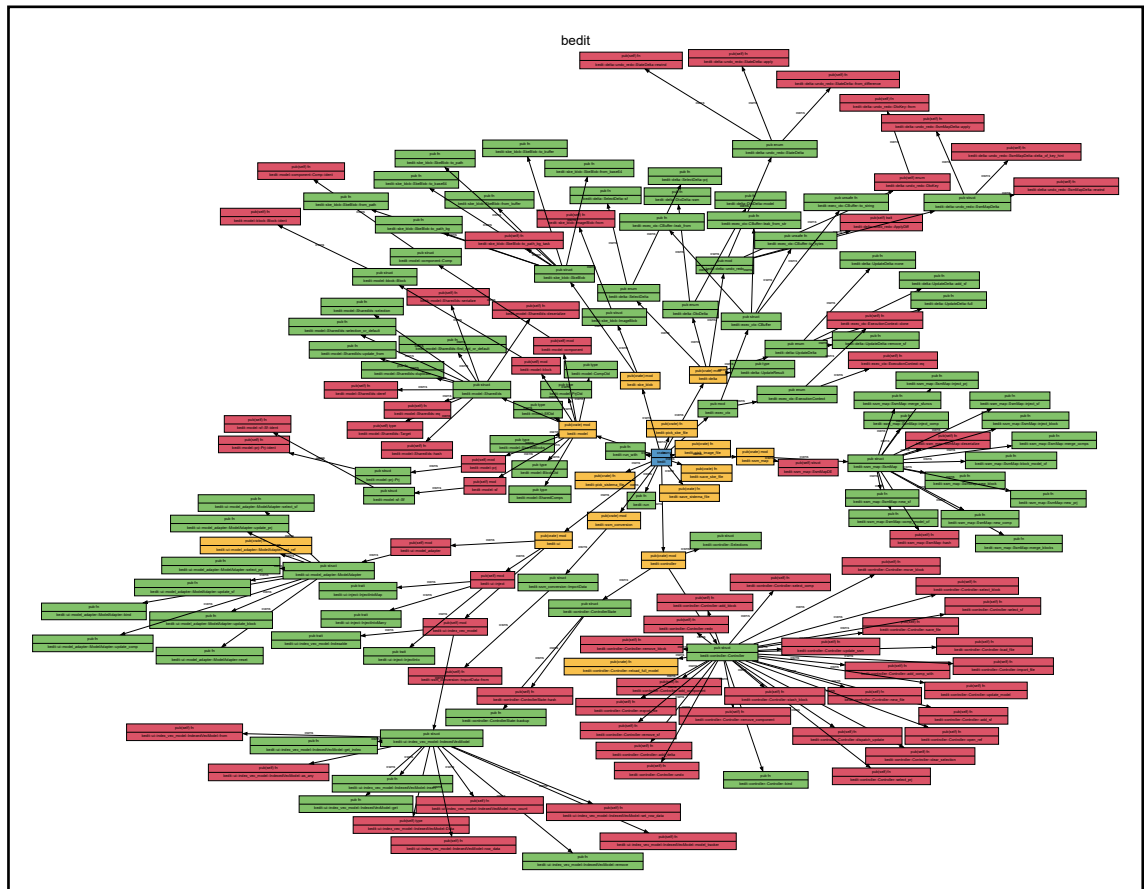


Abbildung 4: Graphbasierte der Übersicht der „bedit“ crate

⁴Diese Graphen sind sehr schwer zu lesen und dienen nur einer Veranschaulichung des Projektumfangs. An Stellen, wo eine Visualisierung der Projektstruktur Sinn macht werde ich eine vereinfachte und besser lesbare Darstellung wählen.

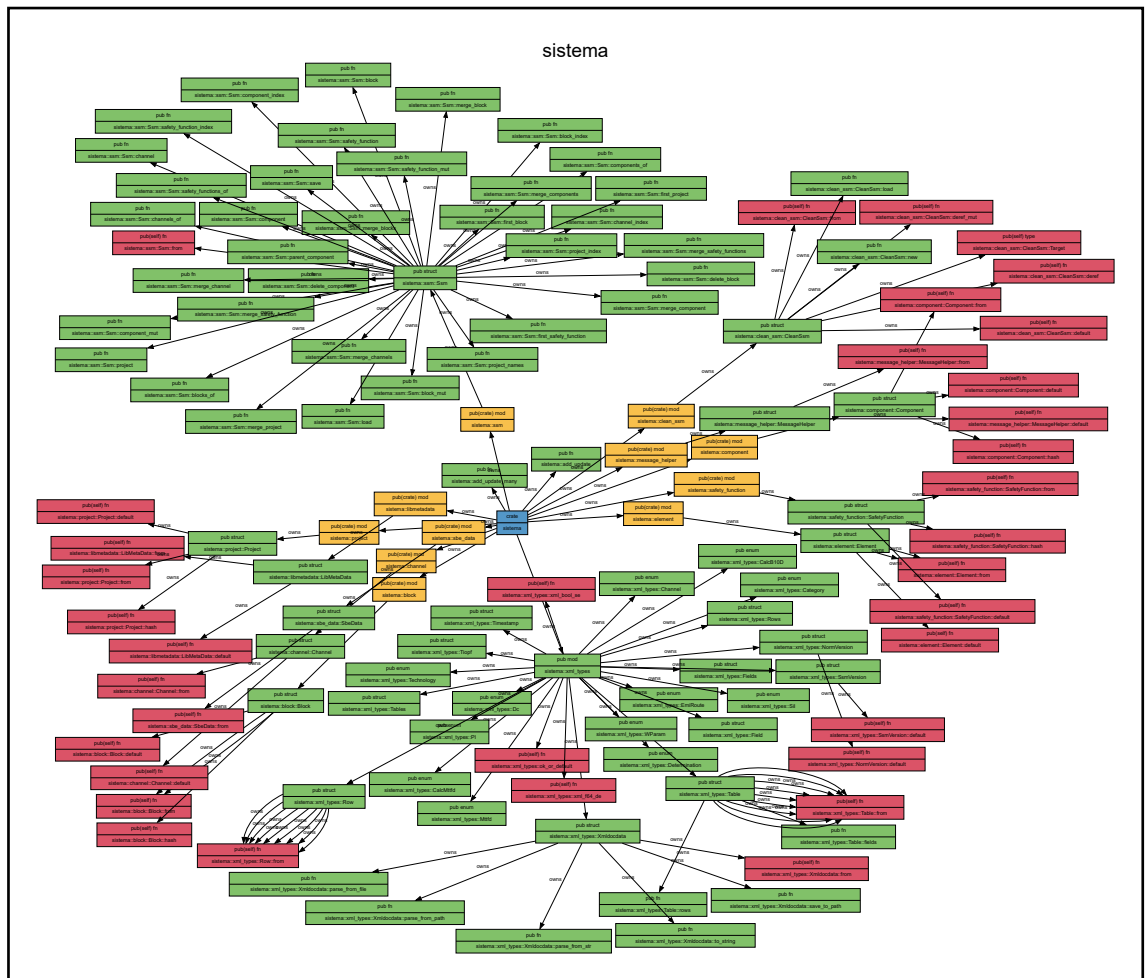


Abbildung 5: Graphbasierte der Übersicht der „sistema“ crate

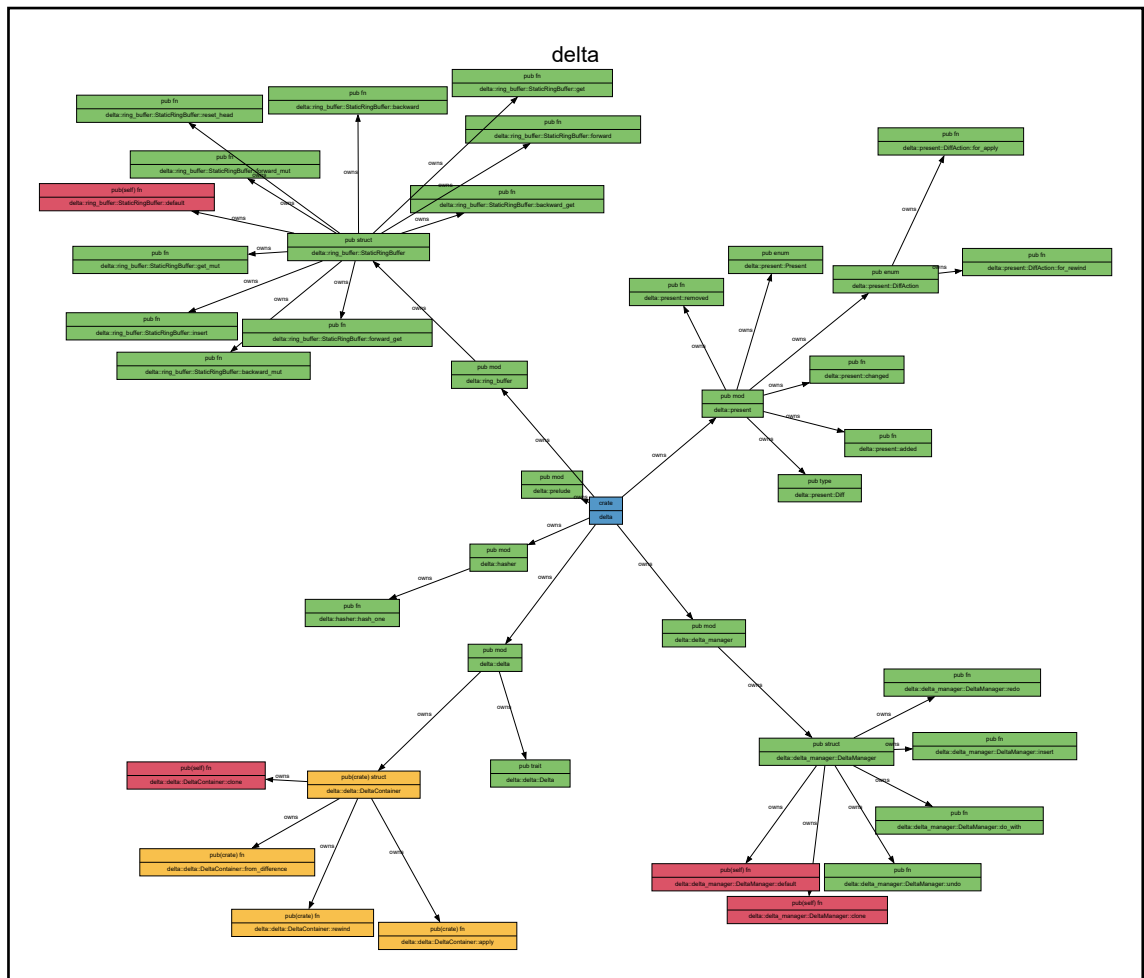


Abbildung 6: Graphbasierte der Übersicht der „delta“ crate

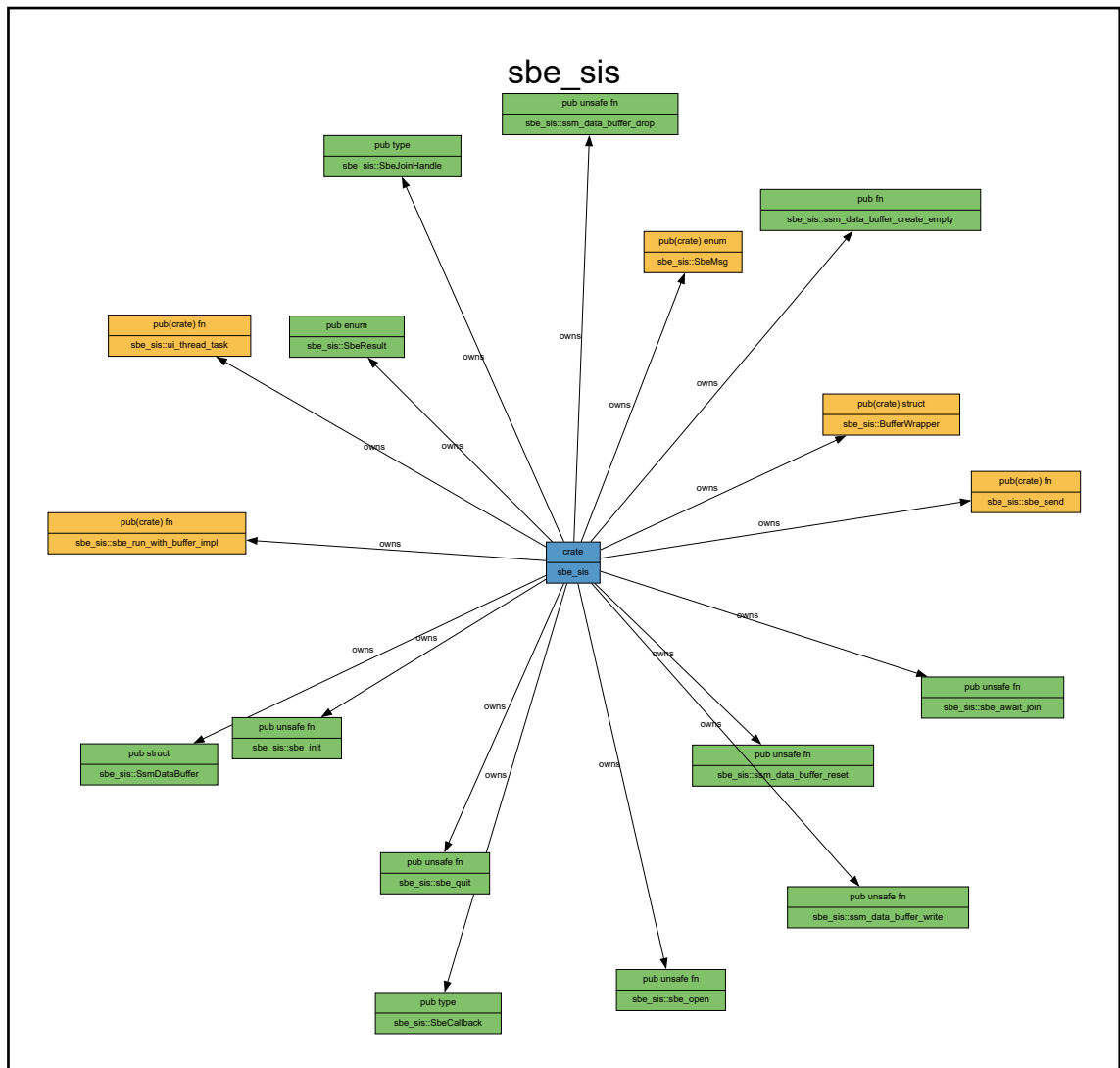


Abbildung 7: Graphbasierte der Übersicht der „sbe_sis“ crate

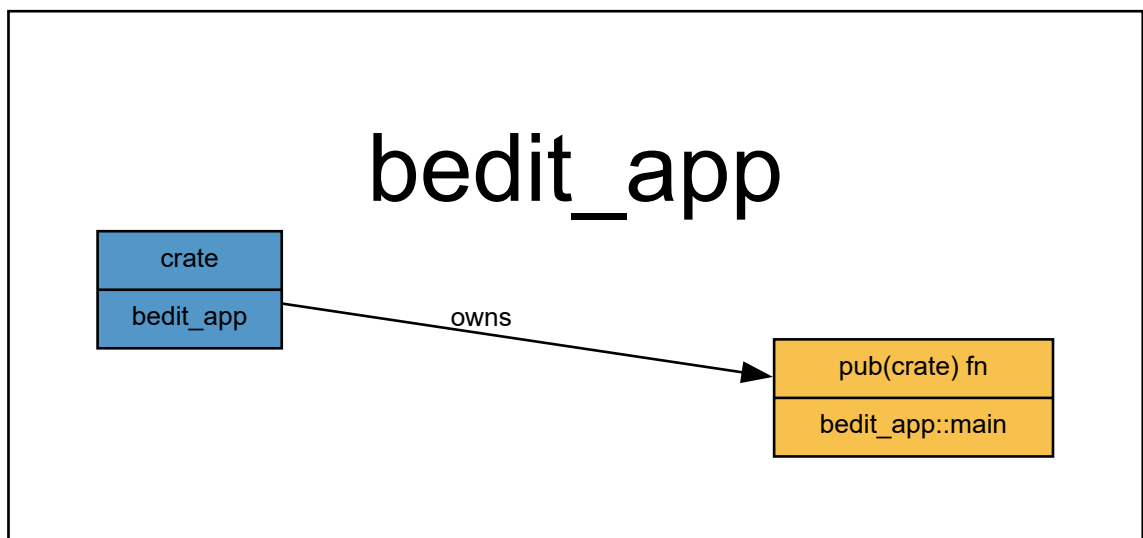


Abbildung 8: Graphbasierte der Übersicht der „bedit_app“ crate

4.3. Architektur der Anwendung

4.3.1. Slint als View im MVC Kontext

Der SBE muss in der Lage sein Daten aus seinem Datenmodell auf der Benutzeroberfläche darzustellen und diese bei Änderung der Daten aktualisieren. Zudem müssen

Änderungen der Daten in der Benutzeroberfläche auch im Datenmodell des SBE reflektiert werden. Dies lässt sich durch ein Model-View-Controller (MVC) Entwurfsmuster[6] elegant lösen.

Der UI-Framework Slint funktioniert gut unter Verwendung dieses Entwurfsmusters, da er keinen direkten Zugriff auf Elemente der Nutzeroberfläche erlaubt, sondern die Verwendung von „Callbacks“ und sogenannten „Properties“ erzwingt.

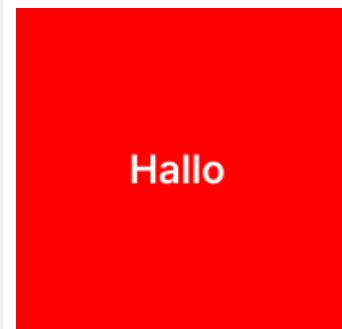
Elemente einer Benutzeroberfläche heißen Komponenten in Slint und werden durch eine Domain Specific Language (DSL) beschrieben, diese kann entweder zur Laufzeit interpretiert oder vor dem Kompilieren der Anwendung kompiliert werden. Eben erwähnte Properties und Callbacks müssen im DSL definiert sein und können damit, für eine Anwendung, die Slint benutzt, das Interface eines Views in einem MVC Kontext beschreiben.

Das folgende Beispiel verdeutlicht die Rolle von Slint-Callbacks und Properties:

Lediglich die grün markierten Stellen können durch den Rustcode verändert oder gelesen werden. Es kann also nicht direkt auf den Text der Text-Komponente im Beispiel zugegriffen werden und auch die TouchArea hat keine Möglichkeit direkt auf das Datenmodell zuzugreifen. Soll der Text verändert werden, müssten Textdaten an das `my_text` Property gesendet werden und um auf einen Klick zu reagieren müsste ein Objekt sich für das `my_callback()` Callback eingetragen haben.

Wenn an ein MVC Muster implementiert werden soll könnte man sich leicht vorstellen, dass ein Controller Struct erstellt wird, dass auf `my_callback()` reagieren und in `my_text` schreiben kann und damit als Brücke zwischen View und Model agiert.

```
1 component MyComponent inherits Window slint
2 {
3     in-out property <string> my_text: "Hallo";
4     callback my_callback();
5
6     Rectangle // Zeigt ein rotes Quadrat an
7     {
8         width: 100px;
9         height: 100px;
10        background: red;
11
12        // Zeigt einen Text an
13        Text {text: my_text;}
14
15        TouchArea
16        {
17            // Reagiert auf einen Mausklick
18            clicked => my_callback();
19        }
20    }
21 }
```



Beispiel 1: Beispiel Slint-Komponente mit einem Callback und einem Property

```

1  #[derive(Default)]
2  struct Model
3  {
4      my_text: String,
5  }
6
7  struct Controller
8  {
9      model: Model,
10     view_adapter: Weak<MyComponent>
11 }
12
13 impl Controller
14 {
15     pub fn new(comp: &MyComponent) -> Self
16     {
17         // register the callback
18         comp.on_my_callback(|| println!("I was clicked"));
19
20         Self
21         {
22             // Store a reference to the view
23             view_adapter: comp.as_weak(),
24             model: Model::default(),
25         }
26     }
27
28     pub fn set_my_text(&mut self, text: String)
29     {
30         self.model.my_text = text;
31         self.view_adapter
32             .unwrap()
33             .set_my_text(
34                 self.model.my_text.to_shared_string()
35             );
36     }
37 }
38
39 fn main()
40 {
41     let view_adapter = MyComponent::new().unwrap();
42     let mut controller = Controller::new(&view_adapter);
43
44     controller.set_my_text(String::from("Bonjour"));
45
46     view_adapter.run().unwrap();
47 }

```

Beispiel 2: Pseudo Rust-Code für einen Controller zu Beispiel 1

SBE implementiert die View Schnittstelle nicht direkt in den einzelnen Komponenten, sondern verwendet dafür sogenannte Globals in Slint [7], das sind Singletons, die sich von allen Komponenten geteilt werden. Die Schnittstelle für Properties ist auf einem Model Global und die Schnittstelle für Callbacks auf einem Actions Global definiert.

Legende:

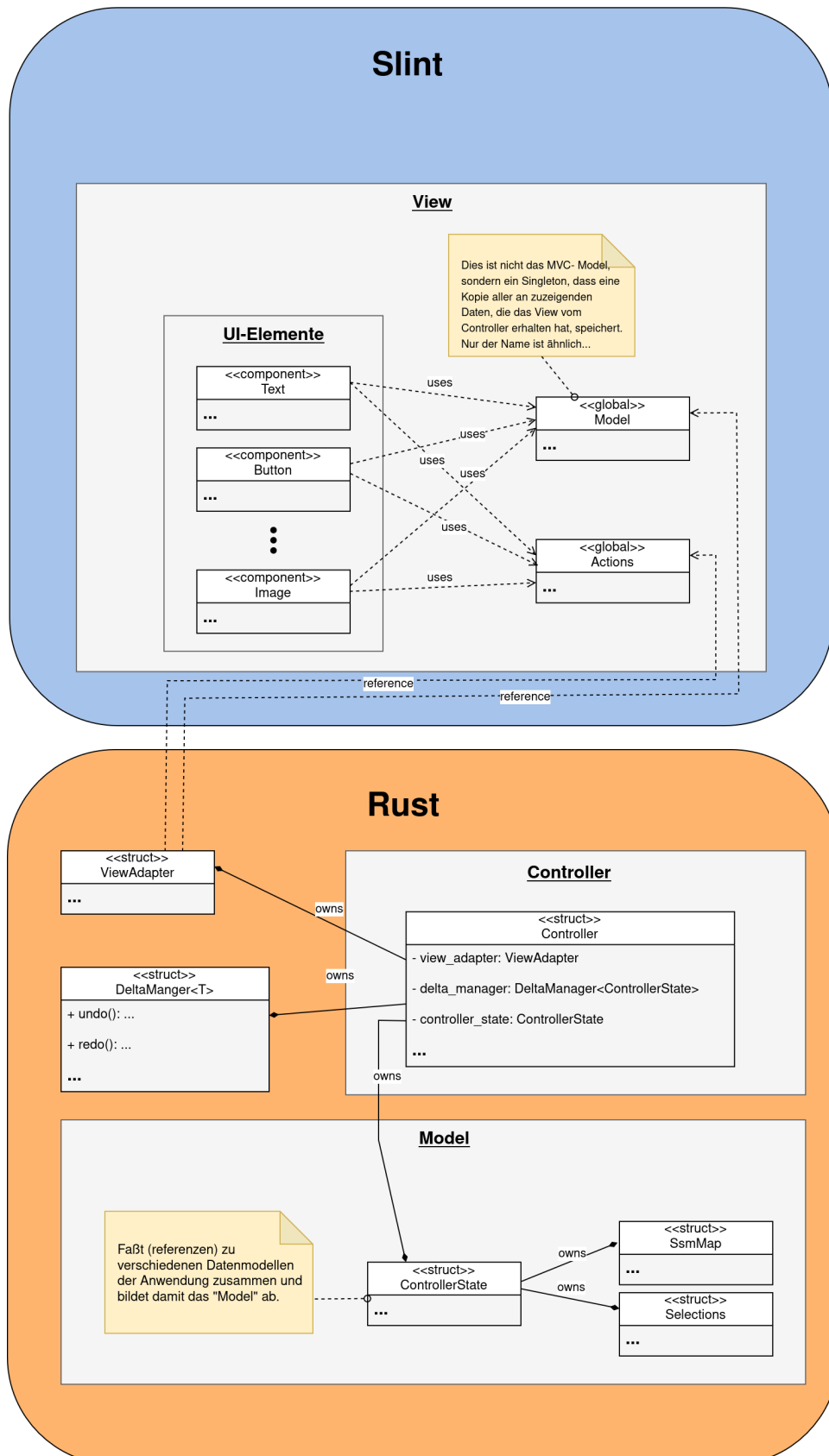
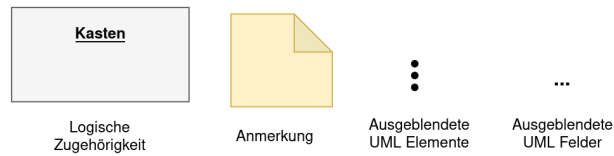


Abbildung 9: SBE Architektur Übersicht

5. Bezug zum bisherigen Studium

6. Fachliche und persönliche Erfahrungen

7. Bibliografie

- [1] „Jahresbericht 2024 Kennzahlen, Themen, Ergebnisse“, März 2025. [Online]. Verfügbar unter: <https://publikationen.dguv.de/widgets/pdf/download/article/5127>
- [2] DGUV, *Wir über uns*. DGUV Website, aufgerufen am 30.06.2026. [Online]. Verfügbar unter: <https://www.dguv.de/de/wir-ueber-uns/index.jsp>
- [3] DGUV, „Satzung des Verbandes Deutsche Gesetzliche Unfallversicherung e.V. (DGUV)“. [Online]. Verfügbar unter: <https://www.dguv.de/medien/inhalt/wir-ueber-uns/aufgaben/dguv-satzung.pdf>
- [4] DGUV, IFA Projektverzeichnis. [Online]. Verfügbar unter: <https://www.dguv.de/ifa/forschung/projektverzeichnis/ifa5128.jsp>
- [5] „Slint Homepage“. [Online]. Verfügbar unter: <https://slint.dev/>
- [6] M. Fowler, *Patterns of enterprise application architecture*, 1st edition. in Addison-Wesley signature series. Boston: Addison-Wesley, 2003.
- [7] „Slint Dokumentation“. [Online]. Verfügbar unter: <https://docs.slint.dev/1.17.0/docs/slint/guide/language/coding/globals/>